PROJECT STATUS & ROADMAP

iwmQT Trading Terminal

Where the platform stands today, the reasoning behind how each piece was built, what's genuinely left, and the plan for getting there.

Prepared for: Engineering Review / Senior Sign-off

Branch: feature/dev

Date: 27 August 2026

4	11/14	3
CORE SUBSYSTEMS BUILT	RMS CONTROLS LIVE	ROADMAP PHASES

01 — EXECUTIVE SUMMARY

Where things stand

The platform has moved from a UI skeleton with mocked pieces to a working NSE F&O terminal with a real order lifecycle: authentication and role enforcement, a 13-control RMS pre-trade risk engine, a live order-matching engine with partial fills, and every trading/admin screen rebuilt to the GUI spec's AG Grid contract.

This document has three parts:

1. **What's completed** — organized by subsystem, with the reasoning/logic behind how each was built, not just a feature list.
2. **What's remaining** — split into what we can build ourselves right now, and what's genuinely blocked waiting on a broker/venue, an AG Grid Enterprise license, or strategy/business input.
3. **Our approach** — a phased plan for the remaining work, sequenced so nothing gets built twice when real inputs eventually arrive.

> **READING GUIDE** Section headings match the platform's own subsystem boundaries (Auth, RMS, Execution, GUI) so this can be read end-to-end or jumped into by area. Table 5's remaining-work items and Section 6's phases use the same item names, so a row in one maps directly to a phase in the other.

02 — COMPLETED

Foundation: Authentication, Roles & Sessions

Role model

> **LOGIC** Five roles (Super Admin, Company Account, RMS Admin, PM, Trader) are enforced in *two* independent places: client-side route guards decide what a user can even navigate to, and server-side `authorize(...roles)` middleware decides what an API call is allowed to do — so hiding a button in the UI is never the only thing standing between a role and an action it shouldn't have.

This mattered concretely: order placement/cancellation originally had no server-side role check at all, meaning any logged-in role could place a trade via a direct API call even though the UI only exposed that button to Traders. Closed by adding `authorize('TRADER')` to both routes.

Password lifecycle

RULE	LOGIC
Complexity	Regex requiring length, letter, digit, and symbol — checked identically on every entry point (signup, reset, admin-created, self-service change).
Reuse prevention	Every password change compares the new password (via bcrypt) against both the current hash and the immediately-previous one, and rotates `previous_password_hash` on success.
Lockout	5 consecutive failures locks the account; unlocking (self-service reset or admin override) also resets the failed-attempt counter, so a stale counter can't instantly re-trigger the lock.
Forced first-login reset	A `must_change_password` flag gates a real session until the user sets their own password — set on admin-created accounts and admin-forced resets.
Expiry	A non-blocking banner appears 3 days before `password_expires_at`; once actually expired, every protected route redirects to a forced change screen instead of rendering.

Sessions

> **LOGIC** Each login mints a random session id (`sid`), stores it on the user row, and embeds it in the JWT. Every authenticated request re-checks the token's `sid` against the stored value — so a second login (optionally forced) invalidates the first session immediately, without needing a token blocklist. A 15-minute idle timer with a visible 60-second countdown handles inactivity independently of this.

03 — COMPLETED

RMS Pre-Trade Risk Engine

The 14 controls, as they stand today

#	CONTROL	STATUS
1 / 4	Price Check / Trade Price Protection	LIVE
2	Quantity Limit	LIVE
3	Order Value	LIVE
5	Market Price Protection	N/A — Market orders aren't enabled in this build (LIMIT-only by spec)
6	Cumulative Open Order Value	LIVE
7	Net Position vs. Margin	LIVE (proxy formula, noted below)
8	Position Limit	LIVE
9	Trading Limit	ALIASED — see Section 5
10	Exposure Limit (user + global)	LIVE
11	Turnover Limit	LIVE
12	Security-Wise Limit	LIVE
13	Automated Execution	PARTIAL — see Section 5
14	Efficient Price Discovery / Fair Play	N/A — no algo/commodity segment in this build
—	OPS (orders/sec) cap	LIVE — separate from the 14, per spec

Two things built specifically to make this trustworthy, not just functional

LOGIC — REJECTION AUDIT TRAIL Until recently, a blocked order left zero record — only successful fills were ever logged. Every one of the 14 rejection points now writes an `ORDER_REJECTED` audit entry tagged with the exact control that fired (e.g. `CONTROL_8_POSITION_LIMIT`) before responding to the trader. "Who tried to place what, and which control stopped it" is now answerable from the audit log itself.

LOGIC — CONFIG CACHE, NOT A DATA CACHE Order placement used to run 4 separate live queries just to read config that rarely changes (`oms_config`, `banned_scripts`, `kill_switches`, `security_limits`). These are now held in an in-memory cache, refreshed the instant an admin edits any of them — never on a timer, so it's always exactly as fresh as the database. Deliberately *not* cached: live

> numbers like position, exposure, and turnover, which change with every order across every user — caching those would make the risk engine wrong, not just faster, so they stay live queries by design.

Kill switches & ban list

Global and per-user trading halts, plus a symbol ban list — both checked ahead of the 14 numbered controls, both now backed by the same config cache, both requiring a confirmation dialog in the admin UI before activating (the global halt originally had none — a genuine risk given it's the single highest-blast-radius control in the app).

04 — COMPLETED

Execution Engine & Order Lifecycle

Matching logic

> **LOGIC** A BUY LIMIT fills once the market trades at or below the limit price; a SELL fills at or above. The fill happens via a single atomic `UPDATE ... WHERE status IN ('PENDING','PARTIALLY_FILLED')`, hooked into the existing 600ms market-tick loop right after each price broadcast — so a slow database round-trip never delays the live feed itself.

Partial fills

The engine now supports filling less than an order's full quantity. A dedicated `fills` table is the immutable record of what actually executed — one row per fill, never mutated — while `orders.filled_quantity` tracks progress and `PARTIALLY_FILLED` is a real status distinct from fully `PENDING` or `EXECUTED`. Positions, turnover, and the Trade Book all derive from `fills`, not from `orders.status = 'EXECUTED'` — which matters because a partially-filled order (or even one later cancelled) can still have real fills on record that must count.

Order lifecycle events

A separate `order_events` table logs `PLACED` / `EXECUTED` / `CANCELLED` at every real transition, feeding a genuine Order Logs screen — replacing what used to be a fake heartbeat console unrelated to actual orders.

Price feed realism

Two correctness fixes to the mock feed: removed a statistical bias that made prices drift away from their reference indefinitely (fixed with a true zero-mean random walk plus mean reversion toward `prevClose`), and added server-side tick-size enforcement (₹0.05) so a direct API call can't submit a price the exchange would never accept — previously this was only checked in the browser.

05 — COMPLETED

GUI: The AG Grid Overhaul

Every grid screen was checked against the GUI Functional & Technical Specification's contract (light theme, transaction-based live updates, cell-flash, CSV export, the required column set) and brought into compliance:

- **Mechanics** — the worst violation (Open Orders doing a full re-render on every poll) rewritten to the same add/update/remove transaction pattern every other grid uses; cell-flash and CSV export added wherever missing.
- **Theme** — every grid converted from dark to the spec's mandated light theme, via one shared palette instead of five duplicated dark-theme blocks.
- **Six admin screens** converted from plain HTML tables to real AG Grid instances (Security Limits, User & Role Management, Company Account Management, Kill Switch, Audit Log, RMS Risk Limits).
- **RMS Risk Limits** rebuilt from a 10-field settings form into an actual 14-control grid, with live-computed utilisation (updated to read from `fills`, matching the partial-fills model) and edit rights correctly restricted to RMS Admin only (Super Admin/Company Account had editing rights they shouldn't have had).
- **Three screens built from nothing**: Order Logs (real event feed), Trade Book (fills-based, with OMS-wise/Token-wise grouping), and Server/OMS Configuration (a new one-OMS-per-trader model with full CRUD).
- **Net Positions** fixed to show per-trader breakdown for RMS/Super Admin viewers — it previously collapsed every trader into one row per symbol, hiding exactly the visibility an RMS Admin needs.
- **Missing columns added**, several requiring new backend fields: PAN/NNF/NEAT ID, Exchange Order ID, Token, Expiry, Option Type, Strike Price, ASM/GSM surveillance stage.

06 — WHAT'S REMAINING

The Gap List

Split into what we can start building ourselves today, versus what's waiting on something outside engineering's control.

Buildable now — no external input needed

ITEM	WHAT IT NEEDS
Real-time push for orders/fills/positions	Extend the existing market-data WebSocket channel with new event types; replace the current 3-second REST poll on Order Book, Open Orders, Trade Book, Net Positions.
Tick data persistence	Write the mock feed's ticks into ClickHouse (already running, already used for auth events) — proves out the ingestion pattern now, against the current feed, so swapping in a real one later is a source change, not a redesign.
Desk / PM-to-trader assignment model	A small schema addition linking traders to a PM, plus an admin screen to manage it — needed for PM's "aggregated (desk)" view to mean something real; today it shows the same data a wider view would.
Connection pooling (PgBouncer)	Infra/ops configuration, no application code changes.
Backup & retention policy	Point-in-time recovery for Postgres now; a ClickHouse retention policy once tick persistence (above) actually exists.
Control 13 — a first anomaly heuristic	Currently only an open-order-count cap. A first pass (e.g. order-rate spikes, repeated-rejection patterns) can be built now and refined once real usage patterns exist.

Needs a decision, not a system

ITEM	WHAT'S NEEDED
Control 9 vs. Control 11	The spec lists Trading Limit and Turnover Limit as two distinct controls; today they're aliased to the same check. Needs a product/RMS decision on what actually distinguishes them before a second config field is added.
Real RMS limit values	Every limit in `oms_config` currently holds a reasonable-looking default. Real per-role, per-company numbers are a business call, not an engineering one.

Blocked on external input

ITEM	WAITING ON
Real broker/exchange feed	Choice of venue/broker and their API/data contract.
True AG Grid row grouping (OMS/Token)	An AG Grid Enterprise license (a sort-based stand-in is in place and clearly labeled as such today).

Strategy Panel / algo functionality	Strategy definitions from the business — currently a UI stub with no backend, intentionally.
Secrets management (Vault / cloud)	The deployment target — this is usually decided alongside hosting, not before it.
Sub-100ms tick-to-render validation	Can't be meaningfully measured until a real feed exists at real tick rates.

07 — OUR APPROACH

How We'll Build the Rest

Sequenced so nothing gets built twice: the "buildable now" items are ordered so each one either de-risks or directly prepares for what comes after it.

1 Phase 1 — Make what exists observable and real-time

> **WHY THIS ORDER** Real-time push and tick persistence both plug into infrastructure that already exists (the WebSocket channel, ClickHouse) — they're the highest-value, lowest-risk work because nothing new has to be stood up, only wired.

- Add `ORDER_UPDATE` / `FILL_UPDATE` / `POSITION_UPDATE` message types to the existing market-data WebSocket, sent from the same places that currently write to `orders` / `fills` / `order_events`.
- Switch Order Book, Open Orders, Trade Book, and Net Positions from polling to subscribing — the AG Grid transaction-update plumbing already in place needs no change, only the trigger source moves from a timer to a socket message.
- Start writing ticks to ClickHouse from the mock feed generator, using the same MergeTree pattern already proven for `auth_events`.

2 Phase 2 — Close the structural gaps

- Desk/PM-to-trader assignment model, so PM's aggregated view is actually scoped, not just labeled that way.
- First-pass Control 13 anomaly heuristic.
- Bring the Control 9/11 decision and real RMS limit values back to product/RMS for sign-off — these block themselves on an answer, not on more engineering time, so raising them early keeps them off the critical path later.

3 Phase 3 — Production hardening

- Connection pooling and a documented backup/retention policy — standard pre-production hygiene, not urgent for a dev/demo environment but necessary before anything resembling a real deployment.
- Secrets management — timed to whenever a deployment target is chosen, since the right tool depends on that choice.

Held for external input (tracked, not blocking the phases above)

> **DESIGN INTENT** The mock feed already sits behind one narrow interface (`getLtp` / `getExpiry` / `getToken` in `marketData.service.js`), and AG Grid's grouping toggles are isolated to two functions per grid. Both were built this way on purpose — swapping in a real feed or a real Enterprise license later should mean changing what's behind that interface, not redesigning the consumers. The moment a broker is chosen, a

> license is procured, or strategy requirements land, that specific item moves from "held" into whichever phase above it fits.
